Types of Activation Functions in Neural Networks

Activation functions are mathematical functions applied to neurons in a neural network to introduce **non-linearity** and help the model learn complex patterns.

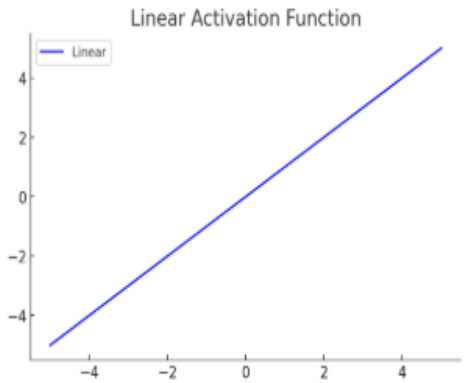
---

1. **Linear Activation Function**

Linear: $f(x) = x$

- Here, **a** is a constant.

Graph:
A straight line passing through the origin with slope **a**.



Advantages:

✅ Simple and easy to use.
✅ Suitable for regression problems.

Disadvantages:

❌ Cannot capture complex patterns (limited learning capacity).
❌ If used in multiple layers, all layers collapse into a single linear function, reducing the power of deep networks.

✅ **Use When:**

- You are solving **regression problems** (output is continuous).
- The model needs to represent a **simple linear relationship**.

❌ **Avoid When:**

- You need a **non-linear decision boundary** (e.g., classification tasks).
- The model is deep, as stacking linear functions results in another linear function (no added complexity).

📌 **Example Use Case:** Predicting house prices based on features like size and location.

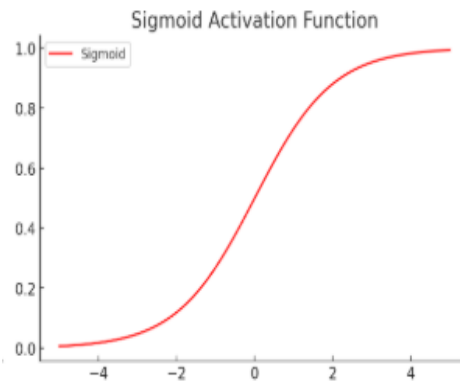
---

2. Sigmoid Activation Function

Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$

- Maps any input between **0 and 1**.

Graph:



- S-shaped (sigmoid curve).

Advantages:

✅ Useful for probabilistic outputs (e.g., binary classification).
✅ Smooth gradient for training.

Disadvantages:

❌ **Vanishing gradient problem** – Small gradients slow down learning in deep networks.
❌ **Not zero-centered** – Can cause slow convergence.

✅ **Use When:**

- Your task is **binary classification** (yes/no, spam/not spam, etc.).
- The output should be **interpreted as probabilities** (values between 0 and 1).

❌ **Avoid When:**

- The model is **deep** (sigmoid suffers from **vanishing gradient** problems).
- The data contains **both positive and negative values** (sigmoid squashes everything between 0 and 1).

📌 **Example Use Case:** Email spam classification (spam or not spam).

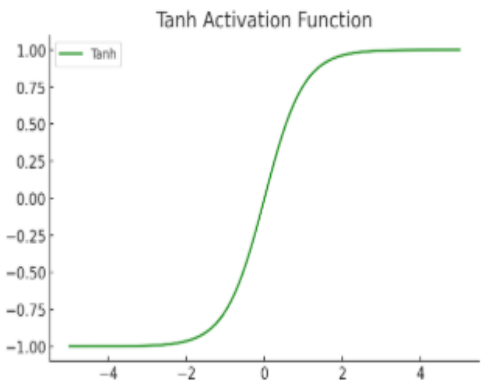
---

3. Tanh (Hyperbolic Tangent) Activation Function

Tanh: $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

- Maps any input between **-1 and 1**.

Graph:



- Similar to sigmoid but centered at 0.

Advantages:

✅ Helps in zero-centered learning.
✅ Stronger gradients than sigmoid.

Disadvantages:

❌ Still suffers from the **vanishing gradient problem**.
❌ Computationally expensive.

✅ **Use When:**

- You need a **non-linear function** that outputs values between -1 and 1.
- The data has **both positive and negative values** (tanh centers data around 0).
- You are working with **shallow networks** (1-2 hidden layers).

❌ **Avoid When:**

- The model is **deep** (like sigmoid, tanh also suffers from vanishing gradients).

📌 **Example Use Case:** Sentiment analysis (positive or negative sentiment).

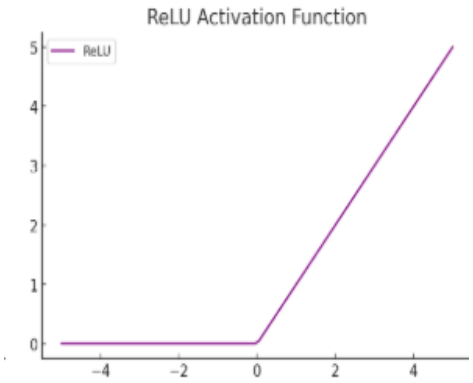
---

4. ReLU (Rectified Linear Unit) Activation Function

ReLU: $f(x) = \max(0, x)$

- Outputs **0** for negative inputs and **x** for positive inputs.

Graph:



- A straight line for **x > 0** and **0** for **x < 0**.

Advantages:

✅ Computationally efficient.
✅ Does not suffer from vanishing gradient (helps in deep learning).

Disadvantages:

❌ **Dying ReLU problem** – Neurons with **x < 0** stop learning (output always 0).

✅ **Use When:**

- You are building a **deep neural network** (most commonly used in hidden layers).
- You want **faster training** (ReLU is computationally efficient).
- Your data contains **both positive and negative values** (negative values become zero).

❌ **Avoid When:**

- Your data has a lot of **zero or negative values** (ReLU outputs 0 for negatives).
- You experience **dying ReLU problem** (many neurons output 0, stopping learning).

📌 **Example Use Case:** Image recognition in deep learning (CNNs).

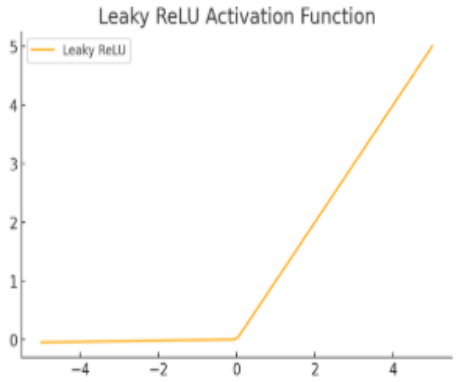
---

5. Leaky ReLU Activation Function

Leaky ReLU: $f(x) = x$ if $x > 0$, else αx

- Allows small negative values instead of **0**.

Graph:



- Similar to ReLU but with a small slope for negative values.

Advantages:

✅ Solves the **dying ReLU problem**.
✅ Retains computational efficiency.

Disadvantages:

❌ Not always differentiable at **x = 0**.

✅ **Use When:**

- You want to **fix the dying ReLU problem** (negative values are given a small slope).
- The data has **negative values that need to be preserved**.

❌ **Avoid When:**

- You are unsure if negative values are necessary for learning.

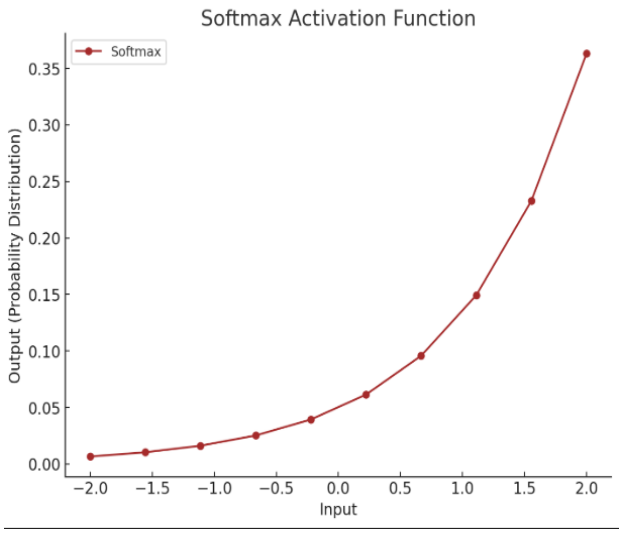
📌 **Example Use Case:** Object detection models where deep networks are used.

---

6. Softmax Activation Function

Softmax: $f(x_i) = \frac{e^{x_i}}{\sum e^{x_i}}$

- Converts values into **probabilities**, useful for **multi-class classification**.



Advantages:

✅ Converts logits into probabilities.
✅ Ensures that probabilities sum up to **1**.

Disadvantages:

❌ Computationally expensive.

✅ **Use When:**

- Your task is **multi-class classification** (3+ classes).

- You need **probabilities** for each class.

❌ **Avoid When:**

- Your task is binary classification (use sigmoid instead).

📌 **Example Use Case:** Classifying handwritten digits (0-9) in MNIST dataset.

Summary of Activation Functions

Activation Function	Range	Used For	Problems
Linear	(-∞, +∞)	Regression	No non-linearity
Sigmoid	(0,1)	Binary classification	Vanishing gradient
Tanh	(-1,1)	Zero-centered outputs	Vanishing gradient
ReLU	[0,∞)	Deep learning	Dying ReLU
Leaky ReLU	(-∞,∞)	Deep learning (fixing ReLU)	Slight computational overhead
Softmax	(0,1)	Multi-class classification	Computationally expensive

1. **Linear Activation Function**: Outputs the input as it is.
2. **Sigmoid Activation Function**: S-shaped curve, maps inputs to a range (0,1), useful for probability estimation.
3. **Tanh Activation Function**: Similar to sigmoid but outputs in the range (-1,1), centered around zero.
4. **ReLU (Rectified Linear Unit)**: Outputs zero for negative inputs and linear for positive inputs, commonly used in deep learning.
5. **Leaky ReLU**: Similar to ReLU but allows small negative values to avoid dead neurons.

Equations of Common Activation Functions

Linear: $f(x) = x$

Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$

Tanh: $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

ReLU: $f(x) = \max(0, x)$

Leaky ReLU: $f(x) = x$ if $x > 0$, else αx

Softmax: $f(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$